

Ball Machine

Η Μαντίνα εφηύρε μια μηχανή με μπάλες για να διασκεδάσει τους συμμετέχοντες της IOI. Η εσωτερική δομή της μηχανής έχει ως εξής:

- Η μηχανή αποτελείται από N **κόμβους**, αριθμημένους από 0 έως και $N - 1$. Ο κόμβος $N - 1$ ονομάζεται **ρίζα**.
- Για κάθε κόμβο u με $0 \leq u < N - 1$, ο **γονέας** του κόμβου u είναι ένας κόμβος $P[u]$ με $P[u] > u$ και ο κόμβος u είναι **παιδί** του $P[u]$. Η ρίζα δεν έχει γονέα. Ένας κόμβος χωρίς παιδιά ονομάζεται **φύλλο**.

Δεν γνωρίζετε την τιμή του N ή τον γονέα $P[u]$ οποιουδήποτε κόμβου u . Αντίθετα, η Μαντίνα σας λέει τον αριθμό των φύλλων, έστω M και ότι τα φύλλα έχουν τους αριθμούς $0, 1, \dots, M - 1$. Ζητείται να βρείτε την εσωτερική δομή της μηχανής, λειτουργώντας την.

Σε κάθε κόμβο της μηχανής μπορεί να βρίσκεται το πολύ μία **μπάλα** και κάθε μπάλα έχει μια **τιμή** η οποία είναι ένας μη αρνητικός ακέραιος αριθμός. Λέμε ότι ένας κόμβος έχει τιμή x αν σε αυτόν βρίσκεται μια μπάλα με τιμή x . Ένας κόμβος είναι **κατειλημμένος** αν σε αυτόν βρίσκεται μια μπάλα. Διαφορετικά, είναι **ελεύθερος**. Αρχικά, κάθε κόμβος είναι ελεύθερος.

Μπορείτε να εκτελέσετε δύο είδη λειτουργιών:

- **insert(U, X)**: επιχειρεί να τοποθετήσει μια νέα μπάλα με τιμή X στο φύλλο U .
 - Εάν το φύλλο U είναι κατειλημμένο, η λειτουργία επιστρέφει `false` χωρίς να εισάγει την μπάλα.
 - Εάν το φύλλο U είναι ελεύθερο, η μπάλα τοποθετείται στον κόμβο U . Στη συνέχεια, η μπάλα μετακινείται επανειλημμένα προς τον γονέα του τρέχοντος κόμβου της, εφόσον αυτός ο γονέας είναι ελεύθερος. Η κίνηση σταματά όταν η μπάλα φτάσει στη ρίζα ή σε έναν κόμβο του οποίου ο γονέας είναι κατειλημμένος. Η λειτουργία επιστρέφει `true`.
- **collect()**: αν η ρίζα είναι ελεύθερη (που σημαίνει ότι η μηχανή είναι κενή), η `collect` επιστρέφει έναν κενό πίνακα. Διαφορετικά, η αναδρομική διαδικασία `traverse` που περιγράφεται παρακάτω συγκεντρώνει τις τιμές για όλες τις μπάλες που βρίσκονται αυτήν τη στιγμή στη μηχανή σε έναν πίνακα S . Αρχικά, ο πίνακας S είναι κενός και γίνεται η κλήση συνάρτηση `traverse(N - 1)`.

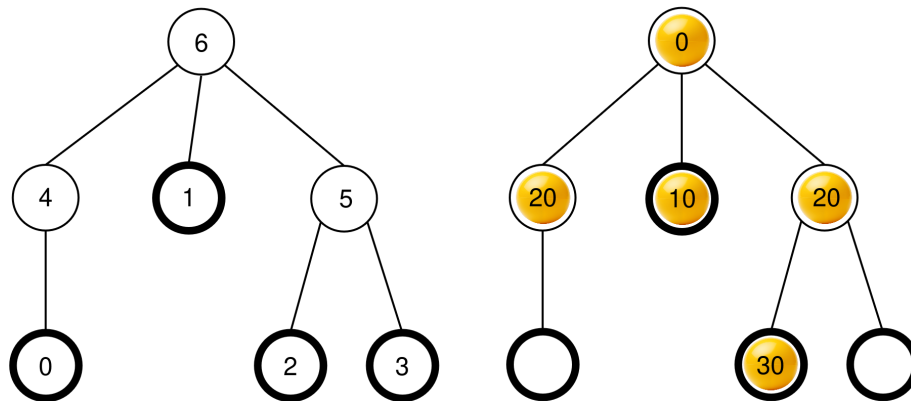
```

traverse(u) :
    πρόσθεσε την τιμή της μπάλας του κόμβου u στο τέλος του S
    έστω c[u] η λίστα των κατειλημμένων παιδιών του u
    ταξινόμησε τη λίστα c[u] σε αύξουσα σειρά των τιμών αυτών
    των κόμβων (αν περισσότεροι κόμβοι έχουν την ίδια τιμή,
    μπορούν να εμφανίζονται με οποιαδήποτε σειρά)
    για κάθε κόμβο v στη λίστα c[u]:
        κάλεσε την traverse(v)

```

Αφού ολοκληρωθεί αυτή η συνάρτηση, **όλες οι μπάλες αφαιρούνται** από το μηχάνημα και η συνάρτηση `collect` επιστρέφει τον πίνακα S .

Για παράδειγμα, θεωρήστε μια μηχανή με $N = 7$ κόμβους και πίνακα γονέων $P = [4, 6, 5, 5, 6, 6]$. Στην αριστερή εικόνα φαίνεται η κενή μηχανή με την αρίθμηση των κόμβων, ενώ στη δεξιά εικόνα φαίνεται μια πιθανή τοποθέτηση από μπάλες, μετά από κάποια ακολουθία λειτουργιών `insert` (αυτή η ακολουθία δίνεται στην ενότητα Παραδείγματα).



Όταν καλείται η συνάρτηση `collect()`, η ρίζα (κόμβος 6) είναι κατειλημμένη, επομένως ο πίνακας S αρχικοποιείται ως $S = []$ και καλείται η `traverse(6)`.

traverse(6): ο κόμβος 6 έχει τιμή 0. Η προσθήκη της στον S δίνει $S = [0]$. Τα παιδιά του κόμβου 6 είναι τα 4, 1 και 5, τα οποία είναι όλα κατειλημμένα. Οι τιμές τους είναι 20, 10 και 20, αντίστοιχα. Η ταξινόμηση σε μη φθίνουσα σειρά δίνει $c[6] = [1, 5, 4]$ (σημειώστε ότι θα ήταν επίσης σωστό αν $c[6] = [1, 4, 5]$, αφού οι κόμβοι 4 και 5 έχουν την ίδια τιμή). Η συνάρτηση στη συνέχεια καλεί την `traverse` για κάθε κόμβο στον $c[6]$ με αυτή τη σειρά.

- **traverse(1):** ο κόμβος 1 έχει τιμή 10. Η προσθήκη της στον S δίνει $S = [0, 10]$. Ο κόμβος 1 δεν έχει κατειλημμένα παιδιά, επομένως αυτή η κλήση τερματίζεται.
- **traverse(5):** ο κόμβος 5 έχει τιμή 20. Η προσθήκη της στον S δίνει $S = [0, 10, 20]$. Ο κόμβος 5 έχει ένα κατειλημμένο παιδί, τον κόμβο 2, άρα $c[5] = [2]$ και η διαδικασία καλεί την `traverse(2)`.

- **traverse(2)**: ο κόμβος 2 έχει τιμή 30. Η προσθήκη της στον S δίνει $S = [0, 10, 20, 30]$. Ο κόμβος 2 δεν έχει κατελημμένα παιδιά, επομένως αυτή η κλήση τερματίζεται.
- Η εκτέλεση επιστρέφει στο **traverse(5)** το οποίο έχει επεξεργαστεί όλα τα παιδιά της λίστας $c[5]$, επομένως αυτή η κλήση τερματίζεται.
- **traverse(4)**: ο κόμβος 4 έχει τιμή 20. Η προσθήκη της στον S δίνει $S = [0, 10, 20, 30, 20]$. Ο κόμβος 4 δεν έχει κατελημμένα παιδιά, επομένως αυτή η κλήση τερματίζεται.

Όλες οι κλήσεις έχουν ολοκληρωθεί και δεν υπάρχουν άλλοι κόμβοι προς επεξεργασία. Τέλος, κάθε μπάλα αφαιρείται από το μηχάνημα και η `collect` επιστρέφει τον πίνακα $S = [0, 10, 20, 30, 20]$.

Ζητείται να βρείτε το πλήθος των κόμβων N και έναν πίνακα γονέων $R = [R[0], R[1], \dots, R[N-2]]$ που να περιγράφει τη δομή της μηχανής, πιθανώς με μια διαφορετική αρίθμηση των κόμβων που δεν είναι φύλλα ούτε η ρίζα. Πιο τυπικά, ένας πίνακας γονέων R θα θεωρείται σωστός εάν σε κάθε κόμβο u της μηχανής είναι δυνατό να αντιστοιχιστεί μια διαφορετική ετικέτα $L[u]$ με $0 \leq L[u] < N$ έτσι ώστε:

- $L[u] = u$ για κάθε $0 \leq u < M$ και για $u = N - 1$, και
- $L[P[u]] = R[L[u]]$ για κάθε $0 \leq u < N - 1$.

Δεν απαιτείται να ισχύει $R[u] > u$. Η μηχανή **πρέπει να είναι άδεια** όταν επιστρέψετε τη λύση σας.

Έστω K το πλήθος των πράξεων `collect` που εκτελούνται και B η μέγιστη τιμή οποιασδήποτε μπάλας σε όλες τις κλήσεις της `insert`. Έστω $C = K + B$. **Η τιμή του C δεν πρέπει να υπερβαίνει το 1000**. Η βαθμολογία σας σε κάποια από τα υποπροβλήματα εξαρτάται από την τιμή του C .

Λεπτομέρειες υλοποίησης

Πρέπει να υλοποιήσετε την ακόλουθη συνάρτηση.

```
std::vector<int> find_structure(int M)
```

- M : το πλήθος των φύλλων της μηχανής.
- Η συνάρτηση καλείται ακριβώς μια φορά για κάθε περίπτωση ελέγχου.
- Η συνάρτηση πρέπει να επιστρέφει έναν σωστό πίνακα γονέων $R = [R[0], R[1], \dots, R[N-2]]$ μήκους $N - 1$.

Για να αλληλεπιδρά με τη μηχανή, η συνάρτησή σας μπορεί να καλεί τις ακόλουθες δύο συναρτήσεις.

Η πρώτη συνάρτηση είναι:

```
bool insert(int U, int X)
```

- U : ο αριθμός του φύλλου όπου τοποθετείται η μπάλα. Πρέπει να ικανοποιείται η συνθήκη $0 \leq U < M$.
- X : η ακέραια τιμή της μπάλας. Πρέπει να ικανοποιείται η συνθήκη $0 \leq X \leq 1000$.
- Αυτή η συνάρτηση επιστρέφει `true` εάν η μπάλα εισαχθεί με επιτυχία ή `false` εάν το φύλλο U ήταν ήδη κατειλημμένο.
- Αυτή η συνάρτηση μπορεί να κληθεί το πολύ 500 000 φορές.

Η δεύτερη συνάρτηση είναι:

```
std::vector<int> collect()
```

- Συλλέγει τις τιμές από όλες τις μπάλες, αδειάζει το μηχάνημα και επιστρέφει τον πίνακα S .

Εάν σε οποιοδήποτε σημείο κατά την εκτέλεση του προγράμματός σας η τιμή του C υπερβεί το 1000, η λύση σας θα λάβει την ένδειξη `Output isn't correct: Too many resources used`.

Η δομή της μηχανής καθορίζεται πριν κληθεί η συνάρτηση `find_structure`. Ο βαθμολογητής είναι **ντετερμινιστικός**, με την έννοια ότι αν τον τρέξετε δύο φορές και εκτελεστεί η ίδια ακολουθία λειτουργιών και τις δύο φορές, οι κλήσεις της `collect` θα επιστρέψουν τους ίδιους πίνακες.

Περιορισμοί

- $2 \leq N \leq 1000$
- $1 \leq M \leq 200$
- $M < N$

Υποπροβλήματα

Υποπρόβλημα	Βαθμολογία	Περαιτέρω περιορισμοί
1	5	Η ρίζα έχει ακριβώς M παιδιά.
2	10	$M \leq 3$
3	25	$N \leq 200, M \leq 45$
4	60	Χωρίς περαιτέρω περιορισμούς.

Στα υποπροβλήματα 3 και 4, η βαθμολογία σας εξαρτάται από την τιμή του C ως εξής.

Υποπρόβλημα 3 (25 βαθμοί)

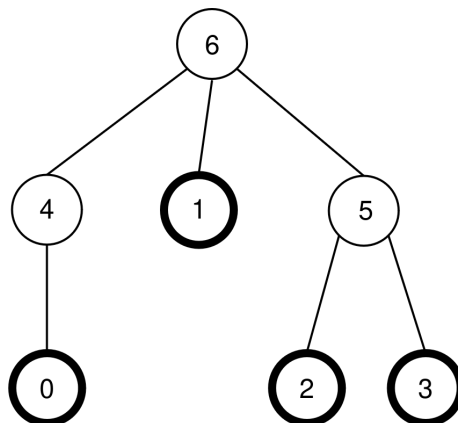
Όριο	Βαθμολογία
$1000 < C$	0
$45 < C \leq 1000$	13
$C \leq 45$	25

Υποπρόβλημα 4 (60 βαθμοί)

Όριο	Βαθμολογία
$1000 < C$	0
$200 < C \leq 1000$	7
$71 < C \leq 200$	$47 - \frac{C}{5}$
$44 < C \leq 71$	$104 - C$
$C \leq 44$	60

Παράδειγμα

Θεωρήστε την ίδια μηχανή που περιγράφεται στην αρχή της εκφώνησης, δηλαδή $N = 7$, $M = 4$ και $P = [4, 6, 5, 5, 6, 6]$. Η μηχανή φαίνεται και πάλι στην εικόνα που ακολουθεί:



Ο βαθμολογητής κάνει την εξής κλήση:

```
find_structure(4)
```

Η συνάρτηση μπορεί να κάνει την εξής ακολουθία κλήσεων:

1. **insert(0, 0)**: το φύλλο 0 είναι ελεύθερο, άρα μια μπάλα με τιμή 0 τοποθετείται στο φύλλο 0. Ο κόμβος $P[0] = 4$ είναι ελεύθερος, άρα η μπάλα μετακινείται στον κόμβο 4. Ο

κόμβος $P[4] = 6$ είναι ελεύθερος, άρα η μπάλα μετακινείται στον κόμβο 6. Δεδομένου ότι ο κόμβος 6 είναι η ρίζα, η κίνηση σταματά. Η κλήση επιστρέφει `true`.

2. `insert(3, 20)`: το φύλλο 3 είναι ελεύθερο, άρα μια μπάλα με τιμή 20 τοποθετείται στο φύλλο 3. Ο κόμβος $P[3] = 5$ είναι ελεύθερος, άρα η μπάλα μετακινείται στον κόμβο 5. Εφόσον ο κόμβος $P[5] = 6$ είναι κατειλημμένος, η κίνηση σταματά. Η κλήση επιστρέφει `true`.

3. `insert(1, 10)`: το φύλλο 1 είναι ελεύθερο, άρα μια μπάλα με τιμή 10 τοποθετείται στο φύλλο 1. Καθώς ο κόμβος $P[1] = 6$ είναι κατειλημμένος, η μπάλα παραμένει στον κόμβο 1. Η κλήση επιστρέφει `true`.

4. `insert(2, 30)`: το φύλλο 2 είναι ελεύθερο, άρα μια μπάλα με τιμή 30 τοποθετείται στο φύλλο 2. Καθώς ο κόμβος $P[2] = 5$ είναι κατειλημμένος, η μπάλα παραμένει στον κόμβο 2. Η κλήση επιστρέφει `true`.

5. `insert(1, 25)`: το φύλλο 1 είναι κατειλημμένο, άρα η μπάλα δεν τοποθετείται στο φύλλο 1. Η κλήση επιστρέφει `false`.

6. `insert(0, 20)`: το φύλλο 0 είναι ελεύθερο, άρα μια μπάλα με τιμή 20 τοποθετείται στο φύλλο 0. Ο κόμβος $P[0] = 4$ είναι ελεύθερος, άρα η μπάλα μετακινείται στον κόμβο 4. Καθώς ο κόμβος $P[4] = 6$ είναι κατειλημμένος, η κίνηση σταματά. Η κλήση επιστρέφει `true`.

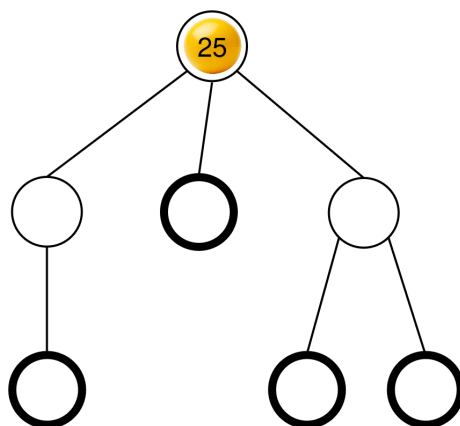
Η τελική θέση κάθε μπάλας φαίνονται ήδη στην εικόνα, στην αρχή της εκφώνησης.

Στη συνέχεια, η συνάρτηση μπορεί να καλέσει:

```
collect()
```

Η εκτέλεση αυτής της κλήσης έχει ήδη εξηγηθεί νωρίτερα στην εκφώνηση, επομένως αυτή η κλήση επιστρέφει $S = [0, 10, 20, 30, 20]$. Στη συνέχεια, η μηχανή είναι και πάλι άδεια.

Στη συνέχεια, η συνάρτηση μπορεί να καλέσει την εντολή `insert(2,25)`. Το φύλλο 2 είναι ελεύθερο, επομένως μια μπάλα με τιμή 25 τοποθετείται στο φύλλο 2. Αυτή η μπάλα μετακινείται στη συνέχεια στον κόμβο 5 και στη συνέχεια στον κόμβο 6, όπου και σταματά. Η κλήση επιστρέφει την τιμή `true`. Η τελική θέση της μπάλας φαίνεται στην ακόλουθη εικόνα.



Τέλος, η συνάρτηση μπορεί να καλέσει τη συνάρτηση `collect()` που επιστρέφει τον πίνακα $S = [25]$ και αδειάζει τη μηχανή.

Υπάρχουν δύο σωστοί πίνακες γονέων που μπορεί να επιστρέψει η `find_structure`: $R = P = [4, 6, 5, 5, 6, 6]$ (που αντιστοιχεί στον πίνακα ετικετών $L = [0, 1, 2, 3, 4, 5, 6]$), και $R = [5, 6, 4, 4, 6, 6]$ (που αντιστοιχεί στον πίνακα ετικετών $L = [0, 1, 2, 3, 5, 4, 6]$). Σε αυτό το παράδειγμα, $K = 2$ και $B = 30$, άρα $C = 32$.

Ενδεικτικός βαθμολογητής

Μορφή εισόδου:

```
N M
P[0] P[1] P[2] ... P[N-2]
```

Μορφή εξόδου:

```
K B C
Q
R[0] R[1] R[2] ... R[Q-1]
```

Εδώ, Q είναι το μήκος του πίνακα R που επιστρέφει η `find_structure`.